

PARCIAL UNIFICADO

Fundamentos de Programación - Java

Versión con respuestas y soluciones

Nombre	Jose Enrique
Material	Guía de estudio
Contenido	Consignas completas + respuestas

Instrucciones de uso

- Primero intente resolver la consigna sin mirar la respuesta.
- Luego compare con la solución propuesta.
- En ejercicios de código, cuide la firma del método: CodeRunner suele ser más estricto que un portero de boliche.

Ejercicios con respuestas

Pregunta 1 - Paso de parámetros

Dado el siguiente código en Java:

```
public class Main {  
    public static void main(String[] args) {  
        int a = 10;  
        int b = incrementar(a);  
  
        System.out.println(a + "," + b);  
    }  
  
    public static int incrementar(int decremento) {  
        return ++decremento;  
    }  
}
```

¿Qué muestra por consola al ejecutarse el programa?

- a. 11,11
- b. 10,12
- c. 10,11
- d. 11,10

Respuesta / solución:

c. 10,11

Pregunta 2 - Nombre más largo en un arreglo

Completar una función que recibe como parámetro un arreglo unidimensional de String con nombres de personas. La función debe recorrer el arreglo y devolver el nombre que tenga mayor cantidad de letras.

Firma requerida:

```
private static String masLargo(String[] nombres)
```

Ejemplo:

```
String[] nombres = {"Ana", "Lucia", "Eduardo", "Juan", "Alejandro"};  
System.out.println(masLargo(nombres));
```

Resultado esperado:

```
Alejandro
```

No se debe implementar el método main.

Respuesta / solución:

```
private static String masLargo(String[] nombres) {  
    String mayor = "";  
  
    for (int i = 0; i < nombres.length; i++) {  
        if (nombres[i].length() > mayor.length()) {  
            mayor = nombres[i];  
        }  
    }  
  
    return mayor;  
}
```

Pregunta 3 - Condicional con error

Dado el siguiente código:

```
public class Usando {  
    public static void main(String[] args) {  
        int valor = 10;  
  
        if (valor = 2) {  
            System.out.println("caso 1");  
        } else {  
            System.out.println("caso 2");  
        }  
    }  
}
```

¿Qué ocurre al ejecutar el programa?

- a. Muestra caso 2
- b. El programa se ejecuta y lanza una excepción
- c. El programa tiene un error de compilación
- d. Muestra caso 1

Respuesta / solución:

c. El programa tiene un error de compilación. En Java, en un if se necesita una condición booleana; valor = 2 es una asignación int.

Pregunta 4 - Comparación de cadenas

Dado el siguiente código:

```
public class EjercicioString {
    public static void main(String[] args) {
        String cadena = "UTEC";
        String otro = new String("UTEC");

        String mensaje = (cadena == otro) ? "iguales" : "diferentes";

        if (mensaje.contains("iguales")) {
            System.out.println("Son iguales");
        } else {
            System.out.println("Son diferentes");
        }
    }
}
```

¿Qué imprime por consola?

- a. iguales
- b. Son iguales
- c. diferentes
- d. Son diferentes

Respuesta / solución:

d. Son diferentes. cadena == otro compara referencias, no contenido; por eso mensaje queda "diferentes".

Pregunta 5 - Duplicar números pares

Completar una función llamada duplicarPares. La función recibe un arreglo de enteros y debe devolver un nuevo arreglo con los mismos valores, pero duplicando solamente los números pares. Los números impares deben quedar iguales.

Firma requerida:

```
public static int[] duplicarPares(int[] arr)
```

Ejemplo:

```
int[] arr = {1, 2, 3, 4, 5};
int[] resultado = duplicarPares(arr);
```

Resultado esperado:

```
[1, 4, 3, 8, 5]
```

No se debe modificar el arreglo original.

Respuesta / solución:

```
public static int[] duplicarPares(int[] arr) {
    int[] resultado = new int[arr.length];

    for (int i = 0; i < arr.length; i++) {
        if (arr[i] % 2 == 0) {
            resultado[i] = arr[i] * 2;
        } else {
            resultado[i] = arr[i];
        }
    }

    return resultado;
}
```

Pregunta 6 - Lista creada con Arrays.asList

Dado el siguiente código:

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        String colores[] = {"Rojo", "Azul", "Verde", "Amarillo", "Rosa"};

        List<String> lista = new ArrayList<>();
        lista = Arrays.asList(colores);

        lista.add("Gris");

        System.out.println(colores);
    }
}
```

¿Qué ocurre al ejecutar el programa?

- Muestra una excepción ya que se accede a un índice inexistente.
- Muestra [Rojo, Azul, Verde, Amarillo, Rosa]
- Muestra [Rojo, Azul, Verde, Amarillo, Rosa, Gris]
- Muestra una excepción ya que esta lista no es mutable de la forma en la que se creó.

Respuesta / solución:

d. Muestra una excepción: `Arrays.asList(colores)` devuelve una lista de tamaño fijo; no permite `add()`.

Pregunta 7 - Operaciones con ArrayList

Crear una clase llamada `OperacionesArrayList`. La clase debe manejar internamente una lista de textos usando `ArrayList<String>`.

Debe implementar los siguientes métodos:

```
public void agregarElemento(String elemento)
public void eliminarElemento(String elemento)
public String mostrarLista()
```

- `agregarElemento` agrega un elemento a la lista.
- `eliminarElemento` elimina un elemento de la lista.
- `mostrarLista` devuelve la lista en formato texto.

La clase debe tener un constructor sin parámetros que inicialice la lista vacía. No se debe implementar el método `main`.

Respuesta / solución:

```
import java.util.ArrayList;

public class OperacionesArrayList {
    private ArrayList<String> lista;

    public OperacionesArrayList() {
        lista = new ArrayList<>();
    }

    public void agregarElemento(String elemento) {
        lista.add(elemento);
    }

    public void eliminarElemento(String elemento) {
        lista.remove(elemento);
    }

    public String mostrarLista() {
        return lista.toString();
    }
}
```

Pregunta 8 - Histograma con numerales

Completar el método llamado `numeros`. El método recibe un arreglo de enteros y debe imprimir un histograma usando numerales `#` separados por guiones `-`, según los valores del arreglo.

Firma requerida:

```
public void numeros(int[] arr)
```

Funcionamiento:

1. Por cada número del arreglo se debe imprimir una línea.
2. Si el número es positivo, se imprime el número original, un espacio y la cantidad de numerales correspondiente.
3. Los numerales deben estar separados por guiones.
4. Si el número es cero o negativo, se imprime solamente el número.

Ejemplo:

```
int[] arr = {1, 2, 3, 4};
numeros(arr);
```

Resultado esperado:

```
1 #
2 #-#
3 #-#-#
4 #-#-#-#
```

Ejemplo con negativos:

```
int[] arr = {-1, 3, -2, 5};
numeros(arr);
```

Resultado esperado:

```
-1
3 #-#-#
-2
5 #-#-#-#
```

No se debe implementar el método main.

Respuesta / solución:

```
public void numeros(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] > 0) {
            System.out.print(arr[i] + " ");

            for (int j = 0; j < arr[i]; j++) {
                System.out.print("#");
                if (j < arr[i] - 1) {
                    System.out.print("-");
                }
            }
        } else {
            System.out.print(arr[i]);
        }

        System.out.println();
    }
}
```

Pregunta 9 - Herencia: Persona, Profesor, Estudiante

Dado el siguiente diagrama de clases:

```

Persona
- nombre : String
- cedula : String
- direccion : String
+ toString() : String

Profesor
- carnetSalud : String
- materia : String
+ toString() : String

Estudiante
- codigo : String
- tieneBeca : boolean
+ toString() : String

```

Se pide implementar las clases respetando la relación de herencia. La clase Profesor debe heredar de Persona. La clase Estudiante debe heredar de Persona. Cada clase debe tener sus atributos y sobrescribir el método toString().

Respuesta / solución:

```

public class Persona {
    private String nombre;
    private String cedula;
    private String direccion;

    public Persona(String nombre, String cedula, String direccion) {
        this.nombre = nombre;
        this.cedula = cedula;
        this.direccion = direccion;
    }

    @Override
    public String toString() {
        return "Persona{nombre='" + nombre + "', cedula='" + cedula
            + "', direccion='" + direccion + "'}";
    }
}

public class Profesor extends Persona {
    private String carnetSalud;
    private String materia;

    public Profesor(String nombre, String cedula, String direccion,
        String carnetSalud, String materia) {
        super(nombre, cedula, direccion);
        this.carnetSalud = carnetSalud;
        this.materia = materia;
    }

    @Override
    public String toString() {
        return super.toString() + " Profesor{carnetSalud='" + carnetSalud

```

```

        + "', materia='" + materia + "'}";
    }
}

public class Estudiante extends Persona {
    private String codigo;
    private boolean tieneBeca;

    public Estudiante(String nombre, String cedula, String direccion,
                      String codigo, boolean tieneBeca) {
        super(nombre, cedula, direccion);
        this.codigo = codigo;
        this.tieneBeca = tieneBeca;
    }

    @Override
    public String toString() {
        return super.toString() + " Estudiante{codigo='" + codigo
            + "', tieneBeca=" + tieneBeca + "}";
    }
}

```

Pregunta 10 - Clase Libro

Crear una clase llamada Libro que represente un libro dentro de una biblioteca.

La clase debe tener los siguientes atributos:

```

private String titulo;
private String autor;
private int numeroPaginas;

```

Debe implementar tres constructores:

- Constructor sin parámetros: titulo = "Título desconocido", autor = "Autor desconocido", numeroPaginas = 0.
- Constructor con título y autor. En este caso, numeroPaginas debe inicializarse en 0.
- Constructor con título, autor y número de páginas.

Además, debe implementar los métodos getter:

```

public String getTitulo()
public String getAutor()
public int getNumeroPaginas()

```

No se debe implementar el método main.

Respuesta / solución:

```

public class Libro {
    private String titulo;
    private String autor;
    private int numeroPaginas;

    public Libro() {
        this.titulo = "Título desconocido";
        this.autor = "Autor desconocido";
        this.numeroPaginas = 0;
    }
}

```



```

    }

    public Libro(String titulo, String autor) {
        this.titulo = titulo;
        this.autor = autor;
        this.numeroPaginas = 0;
    }

    public Libro(String titulo, String autor, int numeroPaginas) {
        this.titulo = titulo;
        this.autor = autor;
        this.numeroPaginas = numeroPaginas;
    }

    public String getTitulo() {
        return titulo;
    }

    public String getAutor() {
        return autor;
    }

    public int getNumeroPaginas() {
        return numeroPaginas;
    }
}

```

Pregunta 11 - Clase Perro

Dada la siguiente clase:

```

public class Perro {
    private String nombre;
    private String raza;
    private int edad;

    public Perro(String nombre, String raza, int edad) {
        this.nombre = nombre;
        this.raza = raza;
        this.edad = edad;
    }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getRaza() { return raza; }
    public void setRaza(String raza) { this.raza = raza; }
    public int getEdad() { return edad; }
    public void setEdad(int edad) { this.edad = edad; }
}

```

Indicar cuál de las siguientes afirmaciones es correcta:

- a. La clase tiene un error en el constructor.
- b. La clase tiene un error en el método setRaza().
- c. La clase no tiene errores.

d. La clase tiene un error en el método setEdad().

Respuesta / solución:

c. La clase no tiene errores.

Pregunta 12 - Arreglo de objetos

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int total = 1 * 2;

        Persona[] personas = new Persona[5];

        personas[0] = new Persona("Carlos", 50);
        personas[1] = new Persona("Mario", 20);
        personas[2] = new Persona("María", 30);
        personas[3] = new Persona("José", 50);
        personas[4] = new Persona("Sofía", 40);

        for (Persona p : personas) {
            total += p.getEdad();
        }

        System.out.println(total);
    }
}
```

¿Qué valor se imprime por consola?

- a. 190
- b. 202
- c. 192
- d. 182

Respuesta / solución:

c. 192. total inicia en $1 * 2 = 2$; luego suma $50 + 20 + 30 + 50 + 40 = 190$; total final = 192.

Pregunta 13 - Sumar elementos de un arreglo

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int[] arreglo = {2, 3, 5, 4, 4, 12, 22};

        int sumar = sumArr(arreglo) + 12;

        System.out.println(sumar);
    }

    public static int sumArr(int[] arr) {
```

```
int suma = 0;

for (int n : arr) {
    suma += n;
}

return suma;
}
```

¿Qué imprime por consola?

- a. 12
- b. 60
- c. 64
- d. 62

Respuesta / solución:

c. 64. La suma del arreglo es 52 y luego se suma 12.

Pregunta 14 - Bucle while

Dado el siguiente código:

```
public class EjercicioFuncion {
    public static void main(String[] args) {
        int numero1 = 2, numero2 = 8;
        double resultado = 0;

        while (numero1 > 10) {
            resultado = calcular(numero1, numero2);
            break;
        }

        System.out.println(resultado);
    }

    private static double calcular(int num2, int num1) {
        double resultado = num1 * 2 - num2;
        return resultado;
    }
}
```

¿Qué imprime por consola?

- a. No hay un resultado correcto.
- b. 14.0
- c. -4.0
- d. -14.0

Respuesta / solución:

a. No hay un resultado correcto. El while no entra porque numero1 vale 2 y $2 > 10$ es falso; se imprime 0.0.

Pregunta 15 - Paso de parámetros con int e Integer

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int x = 5;
        Integer y = Integer.valueOf(2);

        multiplicar(x, y);

        System.out.println(x + y);
    }

    public static void multiplicar(int i, Integer j) {
        i *= 10;
        j *= 10;
    }
}
```

¿Qué imprime por consola?

- a. Muestra por consola el valor 5.
- b. No funciona ya que tiene errores.
- c. Muestra por consola el valor 10.
- d. Muestra por consola el valor 7.

Respuesta / solución:

d. Muestra por consola el valor 7. El método no modifica x ni y fuera del método.

Pregunta 16 - Manejo de excepciones

Completar el procedimiento prueba. El procedimiento recibe como parámetros String dato e int[] vector.

Firma requerida:

```
public void prueba(String dato, int[] vector)
```

El procedimiento debe manejar posibles excepciones:

- `ArrayIndexOutOfBoundsException`
- `NullPointerException`
- `Exception`

Si ocurre una excepción porque se accede a un índice inexistente del arreglo, debe mostrar:

```
índice del array fuera de rango
```

Si ocurre una excepción porque el String es null, debe mostrar:

```
String null, no se puede saber su largo
```

Ejemplo:

```
String dato = null;
int[] vector = {4, 12, 2, 5};
prueba(dato, vector);
```

Resultado esperado:

```
indice del array fuera de rango
String null, no se puede saber su largo
```

Otro ejemplo:

```
String dato = "";
int[] vector = {1, 1, 2, 5};
prueba(dato, vector);
```

Resultado esperado:

```
1
0
Llegamos al final ok
```

No se debe implementar el método main.

Respuesta / solución:

```
public void prueba(String dato, int[] vector) {
    try {
        System.out.println(vector[vector[1]]);
    } catch (ArrayIndexOutOfBoundsException e) {
        System.out.println("indice del array fuera de rango");
    } catch (Exception e) {
        System.out.println("error general");
    }

    try {
        System.out.println(dato.length());
        System.out.println("Llegamos al final ok");
    } catch (NullPointerException e) {
        System.out.println("String null, no se puede saber su largo");
    } catch (Exception e) {
        System.out.println("error general");
    }
}
```

Pregunta 17 - LinkedList

Dado el siguiente código:

```
import java.util.LinkedList;

public class Main {
    public static void main(String[] args) {
        LinkedList<String> lista = new LinkedList<>();
```

```

        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("D");
        lista.add("E");

        procesarLista(lista);

        System.out.println("Elemento en la posición 2: " + lista.get(2));
    }

    public static void procesarLista(LinkedList<String> lista) {
        lista.remove(0);
        lista.remove(2);
        lista.add(2, "X");
    }
}

```

¿Qué imprime por consola?

- a. Elemento en la posición 2: X
- b. Elemento en la posición 2: C
- c. Elemento en la posición 2: E
- d. Elemento en la posición 2: D

Respuesta / solución:

a. Elemento en la posición 2: X.

Pregunta 18 - HashSet

Dado un conjunto implementado con HashSet, analizar su comportamiento.

```

import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        HashSet<String> numeros = new HashSet<>();

        numeros.add("dos");
        numeros.add("tres");
        numeros.add("tres");
        numeros.add("cuatro");
        numeros.add("cinco");

        System.out.println(numeros);
    }
}

```

Indicar cuál de las siguientes afirmaciones es correcta:

- a. HashSet permite elementos duplicados y mantiene el orden de inserción.
- b. HashSet no permite elementos duplicados y no garantiza el orden de impresión.

- c. HashSet permite elementos duplicados, pero los ordena automáticamente.
- d. HashSet no permite elementos duplicados y siempre imprime en orden alfabético.

Respuesta / solución:

b. HashSet no permite elementos duplicados y no garantiza el orden de impresión.

Pregunta 19 - while con incremento

Dado el siguiente código:

```
public class Usandowhile {
    public static void main(String[] args) {
        int x = 2, y = 5;

        while (x < y)
            System.out.println("Estamos en el x =" + x++);

        System.out.println("UTEC");
    }
}
```

Indicar qué sucede al ejecutar el programa.

- a. El ciclo se ejecuta 3 veces.
- b. El ciclo se ejecuta 4 veces.
- c. El ciclo no se ejecuta nunca.
- d. El ciclo se ejecuta una sola vez.

Respuesta / solución:

a. El ciclo se ejecuta 3 veces. x toma 2, 3 y 4 dentro del while.

Pregunta 20 - Suma de números pares

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int arr[] = {1, 3, 5, 7, 9, 13, 17};
        int sum = 0;

        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 0) {
                sum += arr[i];
            }
        }

        System.out.println(sum);
    }
}
```

¿Qué imprime por consola?

- a. 14
- b. 0
- c. 3
- d. 5

Respuesta / solución:

b. 0. El arreglo no tiene números pares.

Pregunta 21 - Sobrecarga de métodos

Dado el siguiente código:

```
class Mensaje {
    public void enviarMensaje(String destinatario) {
        System.out.println("Enviando mensaje a: " + destinatario);
    }

    public void enviarMensaje(String destinatario, String mensaje) {
        System.out.println(
            "Enviando mensaje a: " + destinatario +
            " con el contenido: " + mensaje
        );
    }
}

public class Main {
    public static void main(String[] args) {
        Mensaje msg = new Mensaje();

        msg.enviarMensaje("Ana");
        msg.enviarMensaje("Juan", "Hola, ¿cómo estás?");
    }
}
```

¿Qué imprime por consola?

Respuesta / solución:

Salida:

```
Enviando mensaje a: Ana
Enviando mensaje a: Juan con el contenido: Hola, ¿cómo estás?
```

Pregunta 22 - Procesar matriz

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int[][] matriz = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
    }
}
```



```

        int resultado = procesarMatriz(matriz);

        System.out.println("El resultado es: " + resultado);
    }

    public static int procesarMatriz(int[][] matriz) {
        int suma = 0;

        for (int i = 0; i < matriz.length; i++) {
            for (int j = 0; j < matriz[i].length; j++) {
                if (matriz[i][j] % 2 == 0) {
                    suma += matriz[i][j];
                }
            }
        }

        return suma * 3;
    }
}

```

¿Qué imprime por consola?

- a. 135
- b. 45
- c. 54
- d. 60

Respuesta / solución:

d. 60. Suma los pares $2 + 4 + 6 + 8 = 20$ y devuelve $20 * 3$.

Pregunta 23 - Polimorfismo con vehículos

Dado el siguiente código:

```

class Vehiculo {
    public void encender() {
        System.out.println("El vehículo está encendido");
    }
}

class Coche extends Vehiculo {
    @Override
    public void encender() {
        System.out.println("El coche está encendido");
    }
}

class Motocicleta extends Vehiculo {
    @Override
    public void encender() {
        System.out.println("La motocicleta está encendida");
    }
}

```

```
public class Main {
    public static void main(String[] args) {
        Vehiculo miVehiculo = new Vehiculo();
        Vehiculo miCoche = new Coche();
        Vehiculo miMotocicleta = new Motocicleta();

        miVehiculo.encender();
        miCoche.encender();
        miMotocicleta.encender();
    }
}
```

¿Qué imprime por consola?

Respuesta / solución:

Salida:

```
El vehículo está encendido
El coche está encendido
La motocicleta está encendida
```

Pregunta 24 - Operaciones con métodos

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int a = 5;
        int b = 10;
        int c = 15;

        int result = complexOperation(a, b, c);

        System.out.println(result);
    }

    public static int complexOperation(int x, int y, int z) {
        int step1 = add(x, y);
        int step2 = multiply(step1, z);
        int step3 = subtract(step2, x);

        return step3;
    }

    public static int add(int a, int b) { return a + b; }
    public static int multiply(int a, int b) { return a * b; }
    public static int subtract(int a, int b) { return a - b; }
}
```

¿Qué imprime por consola?

- a. 215
- b. 225
- c. 220

d. 200

Respuesta / solución:c. $220. (5 + 10) * 15 - 5 = 220.$ **Pregunta 25 - Pares, impares y múltiplos de 10**

Escribir una función llamada evenOddSum. La función recibe un entero positivo n. Debe recorrer los números desde 1 hasta n, ambos incluidos.

- Imprimir "odd" si el número es impar.
- Imprimir "even" si el número es par.
- Imprimir "evenTen" si el número es múltiplo de 10.

Firma requerida:

```
public static void evenOddSum(int n)
```

Ejemplo:

```
evenOddSum(10);
```

Resultado esperado:

```
odd
even
odd
even
odd
even
odd
even
odd
evenTen
```

No se debe implementar el método main.

Respuesta / solución:

```
public static void evenOddSum(int n) {
    for (int i = 1; i <= n; i++) {
        if (i % 10 == 0) {
            System.out.println("evenTen");
        } else if (i % 2 == 0) {
            System.out.println("even");
        } else {
            System.out.println("odd");
        }
    }
}
```

Pregunta 26 - Ordenamiento burbuja descendente

Completar un procedimiento llamado `ordenamientoBurbuja`. El procedimiento recibe un arreglo de enteros y debe ordenarlo utilizando el algoritmo Bubble Sort. El orden debe ser descendente, es decir, de mayor a menor.

Firma requerida:

```
public static void ordenamientoBurbuja(int[] arreglo)
```

Ejemplo:

```
int[] arreglo = {3, 10, 2};
ordenamientoBurbuja(arreglo);
```

Resultado esperado:

```
[10, 3, 2]
```

- Debe modificar el arreglo recibido.
- Debe usar Bubble Sort.
- No se debe implementar el método `main`.

Respuesta / solución:

```
public static void ordenamientoBurbuja(int[] arreglo) {
    int n = arreglo.length;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arreglo[j] < arreglo[j + 1]) {
                int temp = arreglo[j];
                arreglo[j] = arreglo[j + 1];
                arreglo[j + 1] = temp;
            }
        }
    }
}
```

Pregunta 27 - Suma de bordes de una matriz

Completar una función llamada `sumaBordes`. La función recibe una matriz cuadrada de enteros y un valor booleano llamado `sentidoHorario`.

Firma requerida:

```
public static int sumaBordes(int[][] matriz, boolean sentidoHorario)
```

La función debe recorrer los bordes externos de la matriz y devolver la suma de sus elementos. Si `sentidoHorario` es `true`, el recorrido debe hacerse en sentido horario. Si `sentidoHorario` es `false`, el recorrido debe hacerse en sentido antihorario.

Ejemplo:

```
int[][] matriz = {
    {1, 2},
```

```

    {3, 4}
};

System.out.println(sumaBordes(matriz, true));

```

Resultado esperado:

```
10
```

Otro ejemplo:

```

int[][] matriz = {
    {1, 2, 3, 4, 5},
    {6, 7, 8, 9, 10},
    {11, 12, 13, 14, 15},
    {16, 17, 18, 19, 20},
    {21, 22, 23, 24, 25}
};

```

La suma del borde debe devolver:

```
208
```

No se debe implementar el método main.

Respuesta / solución:

```

public static int sumaBordes(int[][] matriz, boolean sentidoHorario) {
    int n = matriz.length;

    if (n == 0) {
        return 0;
    }

    if (n == 1) {
        return matriz[0][0];
    }

    int suma = 0;

    for (int j = 0; j < n; j++) {
        suma += matriz[0][j];
    }

    for (int i = 1; i < n; i++) {
        suma += matriz[i][n - 1];
    }

    for (int j = n - 2; j >= 0; j--) {
        suma += matriz[n - 1][j];
    }

    for (int i = n - 2; i >= 1; i--) {
        suma += matriz[i][0];
    }
}

```

```
    return suma;
}
```

Pregunta 28 - Herencia: Empleado y Gerente

Crear una clase Empleado y una subclase Gerente.

La clase Empleado debe tener los siguientes atributos:

```
private String nombre;
private double salario;
private int aniosServicio;
```

Debe tener constructores, getters y métodos para calcular el bono anual y el salario anual. La clase Gerente debe heredar de Empleado.

Para un gerente, el bono anual se calcula de la siguiente manera:

```
bono anual = 0.1 * salario * años de servicio
```

El salario anual se calcula:

```
salario anual = salario + bono anual
```

Se pide implementar ambas clases respetando herencia y sobrescritura de métodos cuando corresponda. No se debe implementar el método main.

Respuesta / solución:

```
public class Empleado {
    public String nombre;
    public double salario;
    public int aniosServicio;

    public Empleado() {
        this.nombre = "";
        this.salario = 0;
        this.aniosServicio = 0;
    }

    public Empleado(String nombre, double salario, int aniosServicio) {
        this.nombre = nombre;
        this.salario = salario;
        this.aniosServicio = aniosServicio;
    }

    public String getNombre() { return nombre; }
    public double getSalario() { return salario; }
    public int getAniosServicio() { return aniosServicio; }

    public double calcularBonoAnual() {
        return 0;
    }
}
```

```

    public double calcularSalarioAnual() {
        return 0;
    }
}

public class Gerente extends Empleado {
    public Gerente(String nombre, double salario, int aniosServicio) {
        super(nombre, salario, aniosServicio);
    }

    @Override
    public double calcularBonoAnual() {
        return 0.1 * this.salario * this.aniosServicio;
    }

    @Override
    public double calcularSalarioAnual() {
        return this.salario + calcularBonoAnual();
    }
}

```

Pregunta 29 - Polimorfismo con animales

Dado el siguiente código:

```

class Animal {
    public void hacerSonido() {
        System.out.println("El animal hace un sonido");
    }
}

class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El perro ladra: ¡Guau! ¡Guau!");
    }
}

class Loro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El loro dice: ¡Hola! ¡Hola!");
    }
}

class Gato extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("El gato maúlla: ¡Miau! ¡Miau!");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal miAnimal = new Perro();
        miAnimal.hacerSonido();
    }
}

```

```

        miAnimal = new Loro();
        miAnimal.hacerSonido();

        miAnimal = new Gato();
        miAnimal.hacerSonido();

        Animal otroAnimal = new Animal();
        otroAnimal.hacerSonido();
    }
}

```

¿Qué imprime por consola?

Respuesta / solución:

Salida:

```

El perro ladra: ¡Guau! ¡Guau!
El loro dice: ¡Hola! ¡Hola!
El gato maúlla: ¡Miau! ¡Miau!
El animal hace un sonido

```

Pregunta 30 - Factoriales

Escribir una función llamada factoriales. La función recibe un entero n y debe devolver un arreglo con los factoriales desde 0 hasta n, inclusive.

Firma requerida:

```
public static int[] factoriales(int n)
```

Ejemplo:

```
int[] resultado = factoriales(5);
```

Resultado esperado:

```
[1, 1, 2, 6, 24, 120]
```

Porque:

```

0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120

```

- El arreglo devuelto debe tener tamaño n + 1.
- La posición 0 debe contener el factorial de 0.
- Cada factorial debe calcularse acumulando el resultado anterior.

- No se debe implementar el método main.

Respuesta / solución:

```
public static int[] factoriales(int n) {
    int[] fact = new int[n + 1];
    fact[0] = 1;

    for (int i = 1; i <= n; i++) {
        fact[i] = fact[i - 1] * i;
    }

    return fact;
}
```

Pregunta 31 - Interfaces

Dada una interfaz en Java, analizar las siguientes afirmaciones sobre su funcionamiento. Una interfaz puede contener métodos abstractos y también métodos default.

Ejemplo:

```
public interface Vehiculo {
    void acelerar();
    void frenar();

    default void describir() {
        System.out.println("Yo soy un vehículo.");
    }
}
```

Una clase que implementa la interfaz debe usar la palabra clave implements.

```
public class Auto implements Vehiculo {
    @Override
    public void acelerar() {
        System.out.println("El auto acelera");
    }

    @Override
    public void frenar() {
        System.out.println("El auto frena");
    }
}
```

Indicar cuál de las siguientes afirmaciones es correcta:

- Una clase usa extends para implementar una interfaz.
- Una clase que implementa una interfaz no está obligada a implementar sus métodos abstractos.
- Una interfaz puede tener métodos default, y una clase la implementa usando implements.
- Una interfaz no puede tener ningún método con cuerpo.

Respuesta / solución:

c. Una interfaz puede tener métodos default, y una clase la implementa usando implements.

Pregunta 32 - Mayor elemento de cada fila

Completar un procedimiento que recibe una matriz bidimensional de enteros. El procedimiento debe recorrer cada fila de la matriz y mostrar el mayor elemento presente en esa fila.

Firma requerida:

```
private static void mayorFilas(int[][] matriz)
```

El formato de salida debe ser:

```
Mayor fila :0 es 4
Mayor fila :1 es 8
Mayor fila :2 es 12
```

Ejemplo:

```
int[][] matriz = {
    {1, 2, 3, 4},
    {5, 8, 7},
    {9, 10, 12}
};

mayorFilas(matriz);
```

Resultado esperado:

```
Mayor fila :0 es 4
Mayor fila :1 es 8
Mayor fila :2 es 12
```

- La matriz puede ser cuadrada o escalonada.
- Debe usarse recorrido con bucles anidados.
- No se debe implementar el método main.

Respuesta / solución:

```
private static void mayorFilas(int[][] matriz) {
    for (int i = 0; i < matriz.length; i++) {
        int max = matriz[i][0];

        for (int j = 0; j < matriz[i].length; j++) {
            if (matriz[i][j] > max) {
                max = matriz[i][j];
            }
        }

        System.out.println("Mayor fila : " + i + " es " + max);
    }
}
```

Pregunta 33 - switch: vocal, consonante o carácter especial

Crear un procedimiento que reciba un carácter y determine si es una vocal, una consonante o un carácter especial/no alfabético. Debe utilizarse una estructura switch.

Firma sugerida:

```
public static void clasificar(char caracter)
```

Ejemplos:

```
clasificar('a'); // es vocal
clasificar('b'); // es consonante
clasificar('@'); // no es una letra
```

- Deben reconocerse vocales minúsculas y mayúsculas.
- Deben reconocerse consonantes minúsculas y mayúsculas.
- Todo carácter que no sea letra debe clasificarse como carácter especial.

Respuesta / solución:

```
public static void clasificar(char caracter) {
    switch (caracter) {
        case 'a': case 'e': case 'i': case 'o': case 'u':
        case 'A': case 'E': case 'I': case 'O': case 'U':
            System.out.println("es vocal");
            break;

        default:
            if ((caracter >= 'a' && caracter <= 'z') ||
                (caracter >= 'A' && caracter <= 'Z')) {
                System.out.println("es consonante");
            } else {
                System.out.println("no es una letra");
            }
    }
}
```

Pregunta 34 - Sumar índices de un arreglo

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int suma = 0;

        for (int i = 0; i < arr.length; i += 2) {
            suma += i;
        }

        System.out.println(suma);
    }
}
```

¿Qué imprime por consola?

- a. 90
- b. 6
- c. 150
- d. 10

Respuesta / solución:

b. 6. Se suman los índices 0 + 2 + 4, no los elementos del arreglo.

Pregunta 35 - Función y procedimiento

Indicar la diferencia entre una función y un procedimiento en Java. Analizar los siguientes métodos:

```
public String getNombre() {  
    return nombre;  
}  
  
public void setPrecio(double precio) {  
    this.precio = precio;  
}
```

Indicar cuál de las siguientes afirmaciones es correcta:

- a. Ambos son funciones porque ambos tienen nombre.
- b. getNombre() es una función porque devuelve un valor, y setPrecio() es un procedimiento porque usa void.
- c. setPrecio() es una función porque modifica un atributo.
- d. Ninguno de los dos métodos puede ser usado dentro de una clase.

Respuesta / solución:

b. getNombre() es una función porque devuelve un valor, y setPrecio() es un procedimiento porque usa void.

Pregunta 36 - Constructores sobrecargados

Crear una clase llamada Persona.

La clase debe tener los siguientes atributos:

```
private String nombre;  
private int edad;
```

Debe implementar tres constructores:

- Constructor sin parámetros: nombre = "Desconocido" y edad = 0.
- Constructor que recibe solo el nombre. En este caso, la edad debe inicializarse en 0.
- Constructor que recibe nombre y edad.

Además, debe sobrescribir el método toString() para devolver el siguiente formato:

```
Nombre: Ana, Edad: 0
```

Ejemplo:

```
Persona p1 = new Persona();
Persona p2 = new Persona("Ana");
Persona p3 = new Persona("Juan", 25);

System.out.println(p1);
System.out.println(p2);
System.out.println(p3);
```

Resultado esperado:

```
Nombre: Desconocido, Edad: 0
Nombre: Ana, Edad: 0
Nombre: Juan, Edad: 25
```

No se debe implementar el método main.

Respuesta / solución:

```
public class Persona {
    private String nombre;
    private int edad;

    public Persona() {
        this.nombre = "Desconocido";
        this.edad = 0;
    }

    public Persona(String nombre) {
        this.nombre = nombre;
        this.edad = 0;
    }

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Nombre: " + nombre + ", Edad: " + edad;
    }
}
```

Pregunta 37 - Excepción por índice fuera de rango

Dado el siguiente código:

```
public class Main {
    public static void main(String[] args) {
        try {
            int[] nums = new int[3];
            System.out.println(nums[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: índice fuera de los límites del array");
        }
    }
}
```

```

    }
}

```

¿Qué imprime por consola?

- a. Imprime 0.
- b. Imprime null.
- c. Imprime Error: índice fuera de los límites del array.
- d. El programa no compila.

Respuesta / solución:

c. Imprime: Error: índice fuera de los límites del array.

Pregunta 38 - LinkedList: agregar y modificar elementos

Dado el siguiente código:

```

import java.util.LinkedList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<String> razas = new LinkedList<>();

        razas.add("Pastor");
        razas.add("CaneCorso");
        razas.add(0, "Yorkshire");
        razas.set(1, "Pitbul");

        for (String raza : razas) {
            System.out.print(raza + " - ");
        }
    }
}

```

¿Qué imprime por consola?

- a. Pastor - CaneCorso - Yorkshire -
- b. Yorkshire - Pitbul - CaneCorso -
- c. Pitbul - Yorkshire - CaneCorso -
- d. Yorkshire - Pastor - CaneCorso -

Respuesta / solución:

b. Yorkshire - Pitbul - CaneCorso -

Pregunta 39 - switch avanzado

Dado el siguiente código:

```

public class Main {
    public static void main(String[] args) {

```

```

int msuma = 0;

for (int i = 3; i > 0; i--) {
    int diferencia = "Gato".compareTo("Perro");
    int opcion = diferencia > 0 ? 1 : 0;

    switch (i - opcion) {
        case 0:
            msuma += i;
            break;

        case 1:
            msuma -= i;
            break;

        case 2:
            msuma += i;

        default:
            msuma++;
    }
}

System.out.println(msuma);
}

```

¿Qué imprime por consola?

- a. 3
- b. 4
- c. 5
- d. 6

Respuesta / solución:

a. 3. Atento: con el código escrito, "Gato".compareTo("Perro") es negativo, por lo tanto opcion vale 0.

Pregunta 40 - Patrones básicos de programación

A partir de los ejercicios anteriores, identificar qué patrón de programación se está utilizando en cada caso.

Relacionar cada concepto con su uso correspondiente:

Concepto	Uso
Bucle	Recorrer arreglos, listas o matrices
Colección	Guardar varios elementos en una estructura
Acumulador	Sumar, contar o calcular factoriales
Matriz bidimensional	Recorrer filas y columnas
Condición	Decidir qué acción realizar
Interfaz	Definir un contrato de métodos
Excepción	Controlar errores de ejecución
Objeto	Representar entidades con atributos y métodos
switch	Selección múltiple
Polimorfismo	Ejecutar métodos sobrescritos según el objeto real

Indicar cuál de los siguientes grupos representa mejor los temas principales del parcial:

- a. HTML, CSS, redes y bases de datos.
- b. Iteración, estructuras de datos, métodos, POO y control de flujo.
- c. Sistemas operativos, memoria RAM y hardware.
- d. Diseño gráfico, edición de video y animaciones.

Respuesta / solución:

b. Iteración, estructuras de datos, métodos, POO y control de flujo.

Clave rápida de opción múltiple

Pregunta	Respuesta	Pregunta	Respuesta
1	c	3	c
4	d	6	d
11	c	12	c
13	c	14	a
15	d	17	a
18	b	19	a
20	b	21	
22	d	23	
24	c	29	
31	c	34	b
35	b	37	c
38	b	39	a
40	b		